

AI in the Research Workflow

What I got wrong in May, and how I work with AI now

Andres L. Marin · Dermot O'Brien · Ilyas Tiouassiouine

11/09/2026

What I learned since May

- In May: the longer my instructions, the more accurate the forecast seemed
- In September I ran the same task again with the newest AI models
- There had been a data leak: the model had found a way to cheat, and I did not notice at the time
- A ten-word request now did real work

That is what today is about: how to work with an AI assistant, and how to check its work.



Three things you will leave with

1. **What these tools actually are**, in plain words
2. **One experiment**: what happened when I gave the AI more, and then fewer, instructions
3. **Three things you can try this week**, whether or not you have ever used AI



1. What these tools are

Question: What is actually sitting behind the chat window?

Think of it as a well-read new colleague

- It has read more than any of us, and it answers in seconds
- It sounds confident, and sometimes it is confidently wrong
- It knows nothing about your project until you tell it

You would not send its first draft to a Director-General unchecked. Nor would you hire it only to fix typos.

You will meet it in three forms

- **Chat:** you ask, it answers. The form most people have tried
- **Built into tools you already use:** Word, Outlook, Teams, your browser
- **Assistants that do multi-step work:** read your files, run the analysis, write the report, while you watch

Today's experiment used the third kind.

What sits around the model: the harness

Part	What it does in a research task
The model	Reads your request and proposes what to do
Your request and standing instructions	Say what you want, the limits, and what to hand back
The harness	Runs the loop: lets the model use tools, keeps track of the work, asks you before risky steps
Tools and files	Your data, your documents, the software, the web
Your check	Compares the result with evidence the model did not produce

The harness changed a great deal between May and September. The same model behaves differently inside a better one.

2. One experiment, and what it taught me

Question: Do more detailed instructions still give better work?

One task, asked three ways

The task: standing at midnight on 1 January 2020, predict Germany's electricity use for every hour of the next seven days.

- **Short:** 10 words
- **Medium:** 46 words, asking for a small computer program, a chart and how accurate it was, plus 7 lines of standing instructions
- **Long:** 1,673 words saying which methods to compare and which inputs to use, plus 113 lines of standing instructions

In May, more detail looked like a better forecast

Anthropic's Claude, the May version (Opus 4.7). One attempt per request.

Request	Average error	What came back
10 words	10.8%	Three simple rules of thumb
46 words	5.5%	A proper statistical forecast and a chart
1,673 words	3.4%	A six-method study, a range showing how sure it was, a write-up

The lesson I drew: write more, get more. One of these numbers is not what it seems.

How the winning forecast got its low error

- To predict each hour, the method looked at the electricity use exactly 24 hours before
- For 2 to 7 January those values did not exist yet. A fair forecast uses its own prediction there. May's method used the real numbers
- So for six of the seven days, the forecast was partly copying the answers. 3.4% was not a real forecast

A data analyst knows not to do this. An AI does it unless it knows, or is told, not to.

Telling the AI what not to do

- The May instructions were written the way most people write them. Their list of things not to do did not cover this case
- In my experience, even an assistant told not to cheat sometimes does. Telling it I was watching for exactly that cut it a lot
- In September, the newer Claude did it the fair way on its own, and labelled the peek as a check
- The newest OpenAI model stopped and asked before starting. It prefers to ask and talk than to just get on with it

Newer models catch more of this themselves. You still say what a valid result is, and you answer when they ask.

The same method, with and without the peek

Claude Fable 5.1 in September: one forecast, measured twice

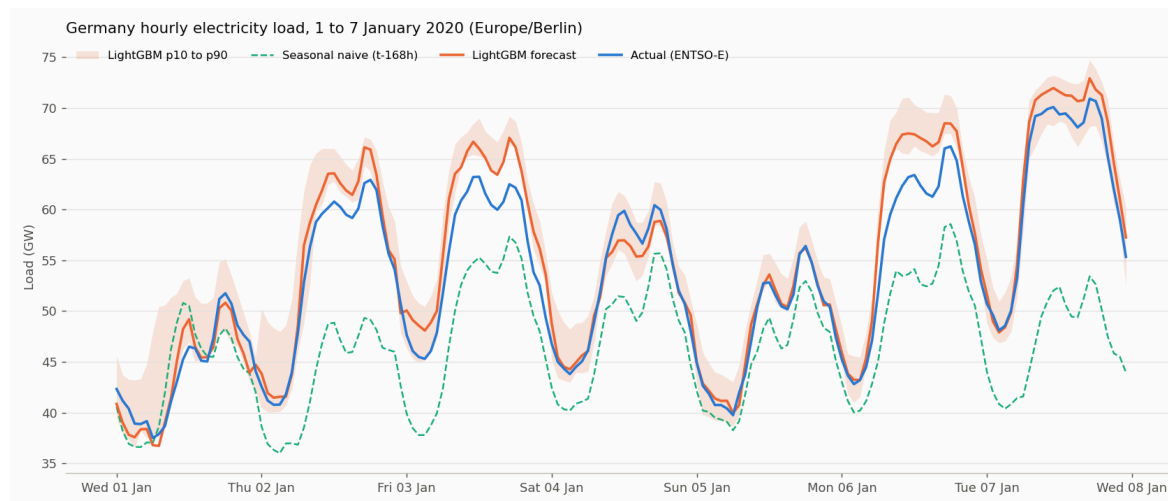
How it filled in the days that had not happened yet	Average error
With the real numbers (the peek)	3.7%
With its own predictions (the fair way)	5.5%

Fable gave the fair 5.5% as its answer and kept the 3.7% only as a check. The May winner gave the peeking number as its answer.

The peek is worth almost two points of error. That, not skill, is why May's 3.4% looked so good.

Ten words in September: a real 3.3%

“Forecast first week of January 2020 German hourly electricity load.”



Claude Fable 5.1, no recipe, no standing instructions. A proper forecast, a range showing how sure it was, and the audit found no answer-sheet use. **Average error 3.3%**, best of three.

With my 1,673-word recipe, the same assistant scored about 5%. The recipe fixed choices it made better on its own.

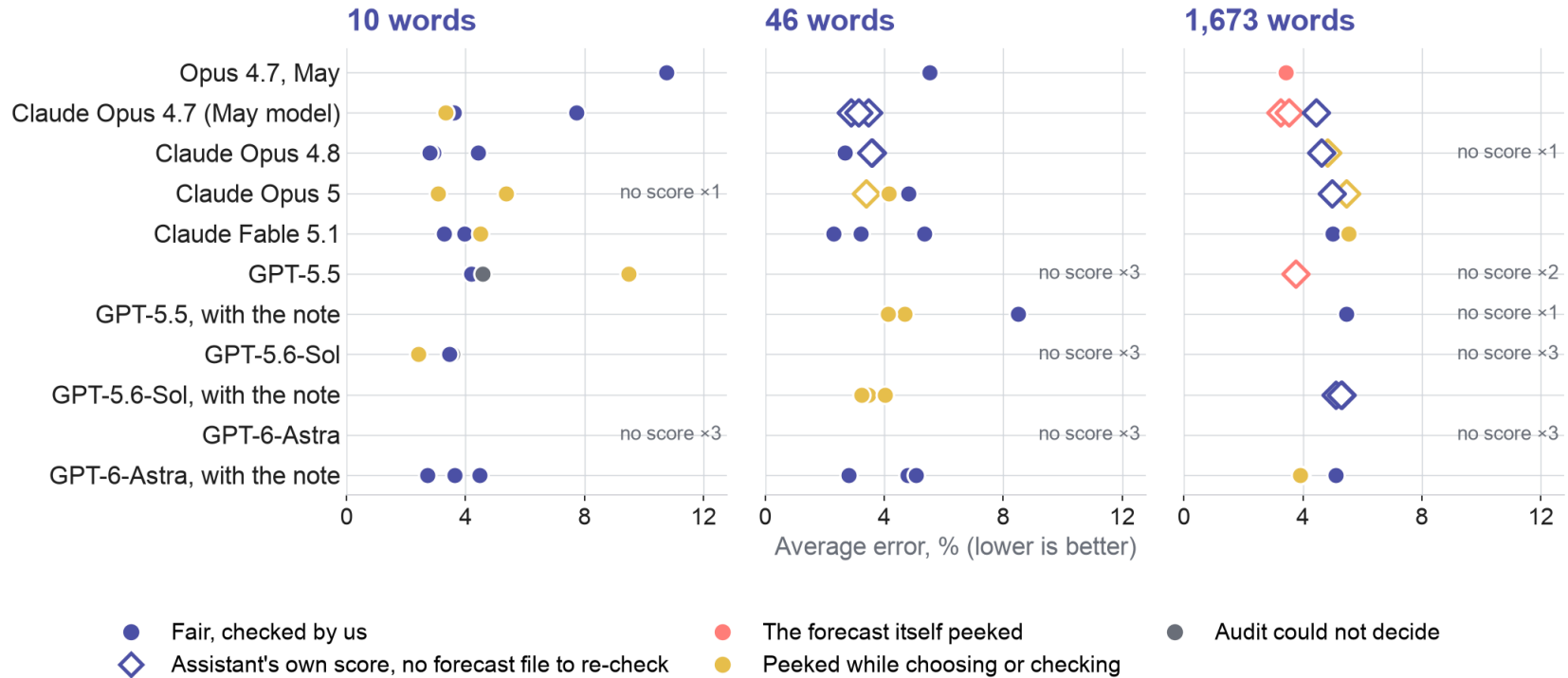
Same model as May, new harness, different answer

- I gave the May model (Opus 4.7) the same ten words again in September, three times
- May: **10.8%**. September: **3.6%**, 3.3% and 7.7%
- Nothing about the model had changed. The harness, its settings and the way I ran it had

The 3.3% attempt adjusted its method after seeing the test score, so it does not count as clean.

A model's result depends on the harness around it. Compare models only inside the same harness, and retest after an upgrade.

One scale for all three request lengths



One dot per attempt. Coral: the forecast peeked. Gold: peeked while choosing.

The recipe did not make the forecasts better. It made them more alike.

It asked the question that exposes the May mistake

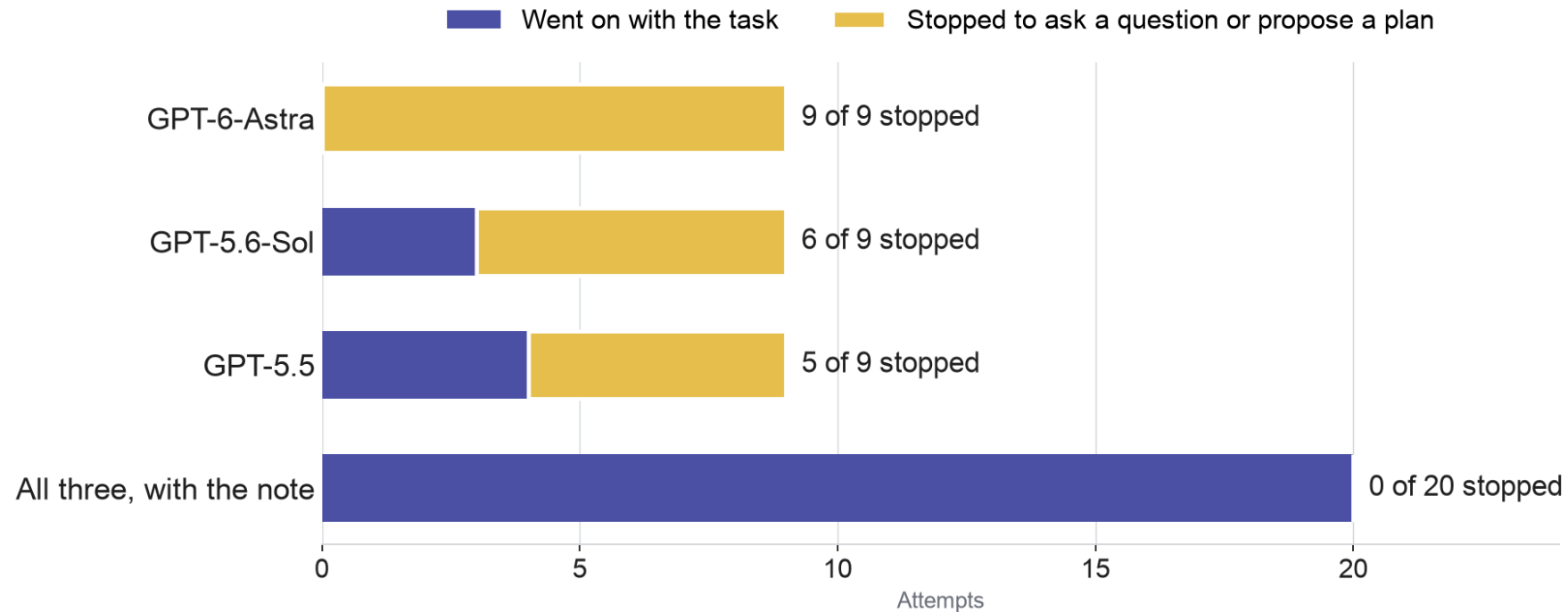
OpenAI's newest assistant, GPT-6-Astra, read the long instructions and asked, in short:

“Do you want one forecast for all 168 hours, made on 1 January, or should I update my inputs as the real numbers arrive?”

That is the exact ambiguity behind the May mistake. Across OpenAI's three models, **20 of 27** attempts stopped to ask or to propose a plan instead of forecasting. Anthropic's Claude asked in 0 of 12 and got on with the work.

A question from the AI is not a weakness. It can be the most valuable thing it produces.

How often the OpenAI models stopped to ask



Different route and settings, not OpenAI's own tool. Original Claude group: 0 of 12 stopped.

With the note, none stopped. Who decides had to be said out loud.

One paragraph, and a different outcome

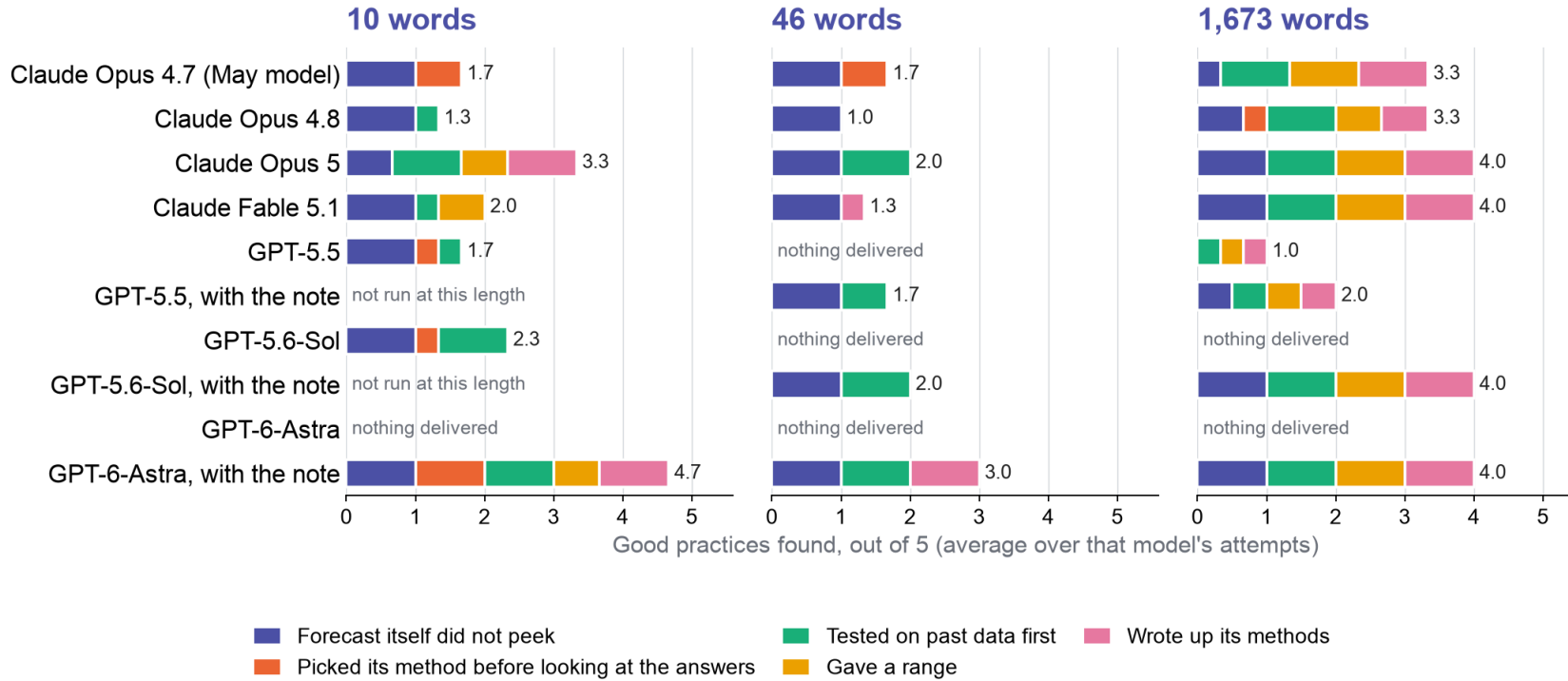
No one could answer, so I re-ran the 20 stopped attempts with this note:

“Note from the operator: this session is unattended. Nobody can answer a question or approve a plan, so do not stop to ask or to propose. Make the most reasonable choice for anything unclear, say what you assumed in your final message, and finish the whole job in this session.”

0 of 20 stopped. **19 of 20** produced a score, 16 of them checked by me. GPT-6-Astra's three ten-word attempts: errors 2.7% to 4.5%, each tested on past years first and written up.

If someone can answer, let it ask. If nobody can, tell it to decide, and to write down what it assumed.

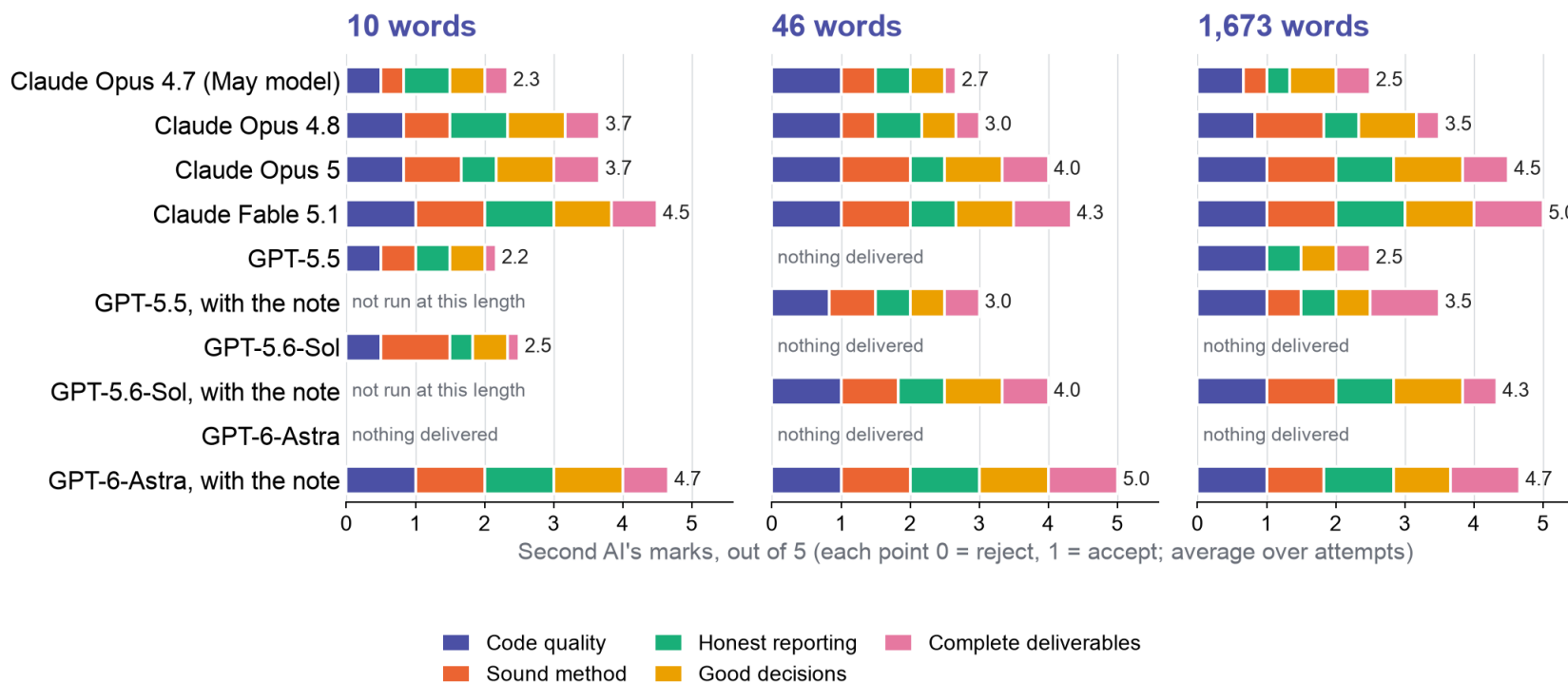
What each request length bought in good practice



Bar length: good practices shown, out of five, averaged over that model's attempts.

Longer instructions bought more good practice. The note bought the most.

I asked a second AI to mark all 62 pieces of work



Five points, 0 reject to 1 accept, averaged over attempts. Not checked by us.

A fast first check, not the last word. It found the peeks the audit found.

3. How I work with AI now

Question: What should you do differently tomorrow?

How I used to work with it: a detailed recipe

- Until this year, I got the best results by spelling out every step: which methods, which inputs, what to produce
- The assistant followed the recipe. Where the recipe was silent, it filled the gap itself, sometimes with a shortcut that made the score look better
- In May, the recipe did not say what to do about days that had not happened yet. So the assistant did not know either

Spelling out every step was the right approach for the older models. With today's models, asking what they would do is often the better start.

How I work with it now: describe the goal, ask, discuss, decide

My experience since the newest models arrived:

- I say what I want to achieve, and what would make the result valid
- I ask how it would approach it, and what it needs to know from me
- It often suggests something I had not considered. Sometimes a better plan
- I choose. It does the work. I check

The skill moved from writing instructions to asking good questions and judging the answers.

Seven questions worth asking every time

1. “Before you start, what questions do you have for me?”
2. “What are three ways to approach this? Which would you choose, and why?”
3. “What could go wrong with this plan?”
4. “Is there a simpler way?”
5. “What am I missing?”
6. “What would an expert in this field say about this?”
7. “Check your own work: what did you assume?”

In my experience, question 1 alone changes your results: it asks for the things you forgot to say.

You are still responsible for the result

- Check the result yourself. Do not take its word for how good it is
- Ask what it assumed, and what it changed after seeing results
- Keep the exact request and data, so someone else can redo it
- A confident tone is not evidence

Its report is a claim. Check it the way you would check a colleague's first draft.

Three things to try on your next piece of research

1. **Before you plan a new study**, describe the question and ask: “How would you approach this, what data would you need, and what could go wrong?”
2. **Give it the papers you have to read anyway**: “Summarise each one, then tell me where they disagree and what you would question.”
3. **Give it a draft section of your paper**: “Where is the argument weakest, and which claims need a source or a check?”

Number one is the one that changes how you work. Ask before you plan, even when you already know how you would do it.

Everything shown today, how to set up the tools, and how to try the experiment yourself: github.com/DermotOBrien-EC/ai-workshop-jrc-2026

Ask, then check

The skill is no longer writing perfect instructions. It is asking good questions and judging the answers.



Slides, records and setup instructions:

github.com/DermotOBrien-EC/ai-workshop-jrc-2026

© European Union 2026. Unless otherwise noted, reuse is authorised under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). Third-party elements retain their respective rights.

Backup and evidence

The evidence behind the talk, for questions. Not part of the 30-minute route.

Of the 83 attempts, 43 have a final score checked by us

- **43** final scores were checked by us: recomputed from the saved forecast
- **17** exist only as the assistant's own number, with no saved forecast we could score
- **23** have no final score at all
- Eight saved forecasts cover a week in local time, one hour off the week the others used; each is scored on the week it actually predicted

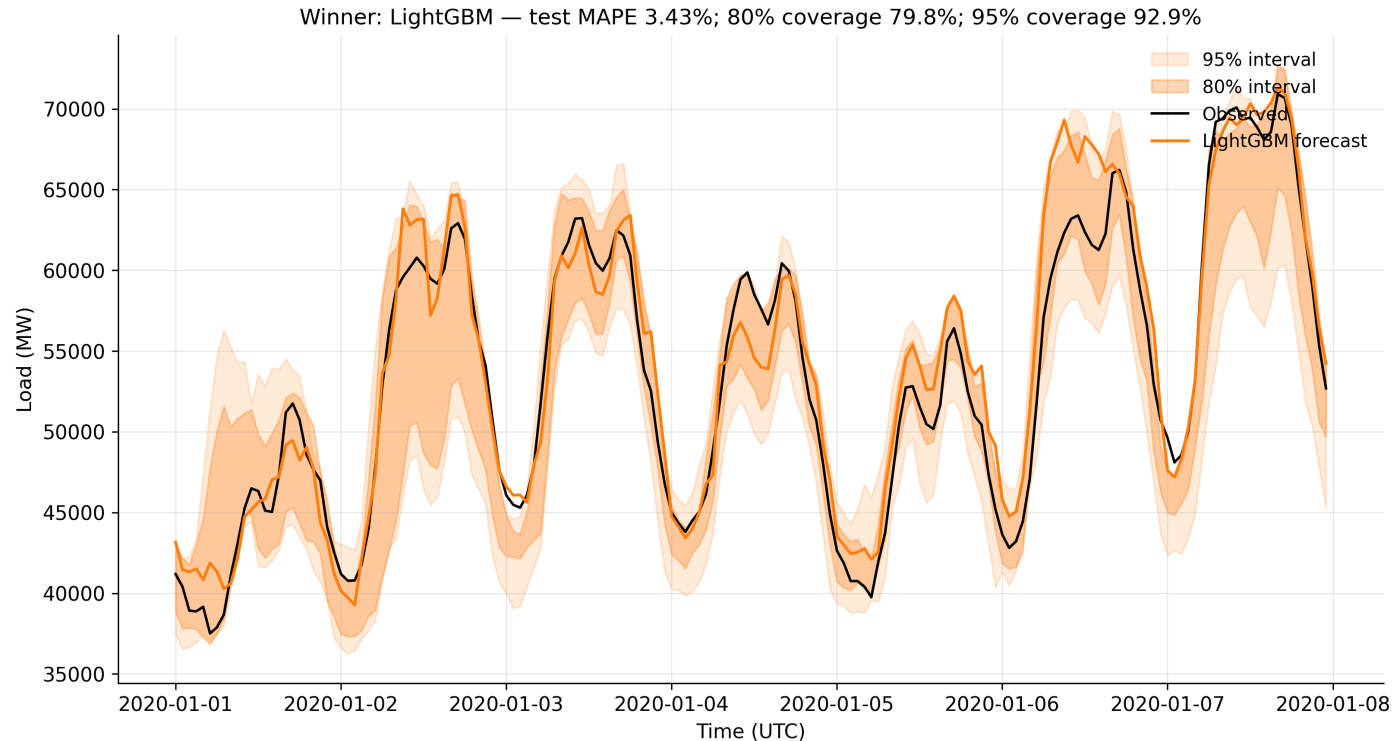
This is a case study describing one forecast week, with few repeats. It cannot say which company's model is better, or prove a general rule for writing instructions.

83 attempts in four groups; the 20 re-runs were the ones that stopped

83 attempts, run on 3–6 September. May's three are separate.

Group	What ran	Attempts
Original rerun	Claude Fable 5.1, three attempts at each of the three requests. Claude Opus 5, one at each	12
A	OpenAI's three models (Astra, Sol, GPT-5.5), through a different route: not OpenAI's own tool	27
B	The 20 attempts from group A that had stopped, re-run with the note that the attempt was unattended and nobody could answer	20
C	More attempts with the May model (Opus 4.7), Opus 4.8 and Opus 5	24

The May winner's chart was made with the peek



The May output, built with real numbers from the week it was predicting. Error 3.43%; its 80% and 95% ranges held the true value 79.8% and 92.9% of the time. Because of the peek, none of that says how a fair forecast would do.

Long instructions can contradict themselves without anyone noticing

- The 1,673-word instructions asked for the winner with the lowest error on the test week
- Elsewhere they said not to use the test week to choose the type of model
- Fable's three 1,673-word attempts followed the first rule
- The same method also won on earlier data in all three: no harm done here, but only by luck

Choose the method on earlier data, and freeze that choice before the final test.

The long recipe did not give Fable the lowest errors

Claude Fable 5.1 in September, three attempts at each request

Request	Average error, in attempt order	What to note
10 words	3.96, 3.28, 4.49%	Attempt 3 is flagged: it changed its inputs after seeing the test score
46 words	2.30, 5.35, 3.20%	Attempt 2 ended before its final step
1,673 words	5.03, 4.99, 5.53%	All three done the fair way: a week-ahead forecast filled with its own predictions

The 10-word attempts were scored on a week in local time, one hour off the week the other attempts used.

The 1,673-word instructions did not give the lowest errors in this comparison.



The long recipe still bought checks, ranges and write-ups

Claude Fable 5.1, counts out of three attempts at each request

Request	A test on past data guided a choice	Gave a range showing how sure it was	Wrote up its methods
10 words	1 / 3	2 / 3	0 / 3
46 words	0 / 3	0 / 3	1 / 3
1,673 words	3 / 3	3 / 3	3 / 3

The 1,673-word recipe asks for these outputs by name. They make the work easier to inspect; they do not show that it is valid.



Giving a range did not make the range reliable

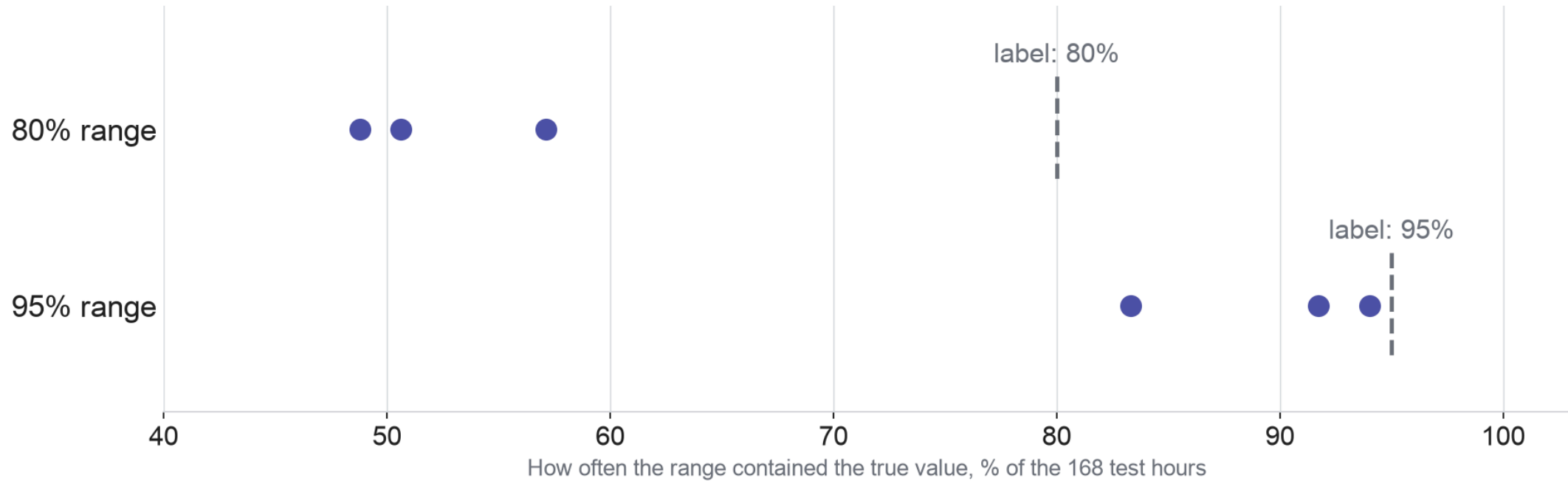


Table 5.1, 1,673-word attempts. Ranges meant to hold the true value 80% of the time held it 57.1%, 50.6% and 48.8% of the time; the 95% ranges 94.0%, 91.7% and 83.3%.

With the note, all three Astra ten-word attempts delivered a forecast

GPT-6-Astra, 10 words plus the note, three attempts

What we looked at	Across the three attempts
Average error	2.71%, 3.63%, 4.47%
A test on past data guided a choice	3 / 3
Wrote up its methods	3 / 3
Gave a range showing how sure it was	2 / 3

The audit records that none of the three attempts scored the week it was predicting. We computed these errors afterwards.



The harness changed mid-experiment, so time and cost cannot be compared

- A keep-alive (to keep a long attempt running) was added for the 1,673-word attempts; the original 10-word and 46-word attempts ran without it
- Some attempts stopped while waiting for a job running in the background; some later attempts were resumed
- The computer sleeping, and jobs running at the same time, distort any comparison of elapsed time
- Installing software, using the internet and occasionally asking a second AI to review mean an attempt did more than the model alone

Whether an attempt finished, how long it took and what it cost need their own accounting. A saved score does not show them.

An upgrade changes behaviour too

- A new version may change how the assistant asks questions, follows old instructions, uses tools, or decides it is done
- OpenAI's current documentation: Astra pauses when standing instructions (what OpenAI calls a "skill") are unclear or conflict
- Anthropic's current documentation: Fable 5.1 changed how it reports progress, when it counts a task as finished, and how closely it sticks to the task

When you move to a new model version, check its behaviour as well as its final answers.

Some instructions last; others need a retest after every upgrade

Always spell out, whatever the model	Check again after a change of model or of harness
The scientific question, and what information the assistant may use	Which software libraries and which modelling recipe to use
Where the data came from, and what must stay confidential	How the task is split into steps and handed from one to the next
The checks a result must pass, and the outputs required	Its settings, and which tools it may use
How far it may go, and what it may do without asking	Workarounds written for an older model's habits

Keep the purpose of a set of standing instructions (what OpenAI calls a “skill”). Change its steps when the evidence shows they need to change.

Before you switch to a new model, run a small check

- Keep a few typical tasks, and write down what a good result looks like
- Write down the model, the harness version, its settings and the tools it can use
- Run your usual work more than once on the new setup, and look at what failed, where it paused, what it produced and what it cost
- Change one relevant instruction or setting, then compare again



Next time, I would write the instructions like this

- Forecast the 168 hours from midnight, 1 January 2020 (UTC), using only what was known at that moment
- Choose the method on earlier periods and fix it before the target week; save predictions, code and a short write-up
- Report how often the range held the true value on earlier periods; we score the target week afterwards
- Nobody will answer: decide the ordinary details yourself, write them down, and say if anything stops you making a valid forecast

Proposed for a future test, not the instructions behind today's results.

Every number in this talk can be traced to these sources

Source	What it holds
Workshop repository	These slides, the numbers behind them, setup instructions for the tools, and how to run the experiment yourself
Experiment repository	The exact requests, the data files and their checksums, the code and the records of every attempt
OpenAI model guidance	How Astra behaves and how to instruct it
Fable 5.1 prompting guide	What changed in its behaviour, and checks to run when moving to it
Anthropic engineering study	Adapting the harness to how the model behaves

Evidence snapshot: **6 September 2026** (WORKSHOP_REVIEW.md records the limitations and where each source is). The charts and the second-AI marks were added on 10 September from the same records.